

Minecraft Building Generator

Nathan Camilleri
University of Malta
nathan.camilleri.19@um.edu.mt

ABSTRACT

Despite its use in various projects and games, procedural content generation is still commonly recognised as an under-researched topic in Artificial Intelligence. This study offers a deeper understanding of *Minecraft* and PCG by defining the problem and thoroughly reviewing the current state-of-the-art research. The proposed solution is a *Minecraft* settlement generator that intends to use procedural content generation to emulate humans in constructing settlements. Our idea is heavily influenced by the GDMC competition, which encourages developers to push this field of study to new heights.

Keywords

Minecraft, Artificial Intelligence, Procedural Content Generation, Computational Creativity, Generative Design

1. INTRODUCTION

Because of its popularity, simplicity of modification, and capacity to tackle complicated tasks in an open-world setting [39], *Minecraft* has lately emerged as an appealing platform for artificial intelligence (AI) research [15]. Consisting of a voxel-based game environment, it is well-suited for controlled AI experiments since researchers can easily access and manipulate precise details without dealing with things such as curvature or deformable objects [26]. A particular area of AI research that can use *Minecraft* as an experimental platform is Procedural Content Generation (PCG). It is an appealing strategy that improves the whole game experience by offering unique content, such as dungeons, levels, and buildings, developed with little or no human involvement [40]. This technique is advantageous because games are becoming more prominent in size and budget, and therefore assisting creators in the development process. Developers may also use such algorithms to generate new game methods, design and develop games, and increase replayability [41].

1.1 Problem Definition

Usually, most PCG algorithms generate content from a blank slate without considering external factors, enabling the developer to concentrate only on the algorithm's output. However, once the algorithm is required to respond to existing content, the situation gets more challenging since the algorithm must adapt to the external content while creating newer content that suits the present conditions. PCG is already present in *Minecraft* in the form of game-generated

NPC settlements.

A competition, known as the Generative Design in *Minecraft* (GDMC) competition [30], was created in 2018 to stimulate further development of PCG utilisation. This competition's primary goal is to enable developers to perform with Adaptive and Holistic PCG to push this field of study towards more remarkable achievements. Developers build an AI artefact capable of generating a functional, attractive, and convincing settlements tailored to a specified *Minecraft* map, which a panel of judges reviews based on specific criteria: adaptability, functionality, evocative narrative, and aesthetics. As a result, it is intriguing to delve deeper into this field of research and attempt to construct our own settlement generator while replicating the GDMC in our study.

1.2 Motivation

The motivation for this project is to study and develop a method for constructing settlements within *Minecraft* using PCG through the objectives stated in the following section. As a result of creating such a technology, settlements will be automated, one-of-a-kind, and visually appealing. We can also comprehend the benefits and drawbacks of the adaptive solution, obtain a greater understanding, and enable future researchers to develop a more robust solution.

As mentioned in the previous section, the GDMC settlement generation competition [30] compelled us to investigate this issue since, in the competition, several solutions compete against one another to determine the best one. When creating such an artefact, referring to this competition is beneficial since it presents a predefined set of criteria that we may replicate while assessing our solution.

1.3 Aims and Objectives

The principle aim of this Final Year Project is to develop an AI system that can emulate humans in constructing settlements within *Minecraft* using procedural content generation (PCG). In order to attain this goal, the following objectives have been identified:

- Build a dataset for the artefact to comprehend conditions such as climate, water bodies, mountainous areas and other aspects of the environment that may impact the algorithm's generation.
- The generated content is adaptable to the current surroundings, where a user may obtain most of the resources needed from the environment in which it is placed.

- Dividing buildings into neighbourhoods where the structures adhere to a consistent yet pleasing architectural and aesthetic style.
- Functionality from an embodied viewpoint, where structures are both accessible and practical.

We comprehend that determining whether the algorithm delivers the anticipated outcomes is subjective. Therefore, we will understand if we have achieved the objectives of this project solely by conducting the later described user study.

2. BACKGROUND RESEARCH

2.1 Minecraft

Minecraft [22] is a voxel-based, open-world game initially developed in Java. Various modes of play are available, including *Minecraft Classic*, which is available for free to play on [21], alongside single-player and multi-player modes present in the paid version of the game. Players can explore a 3D world filled with endless possibilities. One can choose to go mining for various resources, gather numerous materials, craft items, build anything they imagine, indulge in combat against monsters, obtain achievements, and follow the *Minecraft* storyline¹. However, many players ignore such possibilities and decide to play *Minecraft* as just a creative Sandbox, with one of the most common projects being the construction of settlements. Such settlements are assembled through the organisation of blocks, which is the fundamental component within *Minecraft*. While consisting of a simple texture, a block, which can also take the form of a liquid or interactable object, is placed in its discrete three-dimensional coordinate. The vast majority of the blocks are discoverable in the open world, or otherwise, can be crafted by the player with the obtained raw resources [13].

2.1.1 *Minecraft* settlements

Upon creating a world in *Minecraft*, various game elements are spawned together with *Minecraft* settlements - if the option to generate them is triggered. An NPC village is one such example, being a group of game-generated structures randomly placed on the world's surface [30]. Buildings inside the settlement adapt according to the biome they spawn in, with particular block types, characteristics, and features of the generated buildings changing correspondingly. Purposefully, they serve as protection to NPC villagers from dangerous creatures, while also providing farmable goods from the generated farms within the settlement. While such functional features mimic possible villages in the real world, certain shortcomings are present, which include the lack of warmth, sanitation, and physics [13].

2.2 MCEdit

MCEdit² is a free software used as a world editor for *Minecraft*, initially made to provide players with the possibility of maintaining any structure while transitioning from one *Minecraft* version to another. Subsequently, additional valuable features were then implemented, such as loading saved games from various *Minecraft* versions, being able to navigate through a loaded world, showing locations of entities, exporting components, automatically adapting certain

blocks between *Minecraft* versions, editors for interactable blocks, various brush tools, and several more convenient tools. MCEdit is also an excellent environment to delve into AI generative techniques within a *Minecraft* world.

2.2.1 *The GDMC Framework*

The Generative Design in *Minecraft* (GDMC)³ framework, originally designed to accommodate settlement generation in a competition, is a layered structure that utilises MCEdit to test AI within *Minecraft* [30]. Accessing and altering a *Minecraft* map file can be achieved through the 'Filter' option included in MCEdit itself, which is an option that runs commands that can add functionality to MCEdit, or programmatically alter *Minecraft* worlds by reading code and modifying the terrain accordingly. Scripts can incorporate algorithms that can construct complex ideas such as giant forests, volcanoes, oceans, and even procedurally generate entire villages. This framework will be utilised in this project.

2.3 Procedural Content Generation

PCG refers to algorithms used to automate the creation of game material that an individual would typically produce [41]. The additional increase in creativity and adaptivity, alongside the reduction of development time and cost, are among the numeral reasons individuals decide to use this approach [40]. Several games adopted such a technique, with examples being *Rogue* [42], *Astroneer* [36], *Factorio* [35], *No Man's Sky* [11], and *Valheim* [38]. This area of AI includes various applications, with two types of solutions being the Adaptive and Holistic concepts, discussed in the points below.

2.3.1 *Adaptive PCG*

Adaptive PCG refers to the challenge of adapting the generation according to a complex initial input. It is a problem in computational creativity that is perhaps understudied. Another unresolved topic is how to give the generator more authority over high-level decisions and thus, improving their computational creativity. Designers are currently in charge of selecting a high-level concept and converting that theme into elements, including buildings, materials, and decorations [28]. Considering this concept of adaptivity with relation to a *Minecraft* generator, unseen maps would be provided, with the produced settlements interacting with the existing content of the world, such as the landscape and climate. Aspects such as narrative functionalities, buildings, and pathways should all be adaptable to their surroundings [30].

2.3.2 *Holistic PCG*

Ensuring that the many forms of generated content and the numerous dimensions in which content expresses itself are compatible is referred to as holistic PCG. Therefore, factors such as buildings, paths, and natural features are all considered since we are not looking for the generation of one aspect alone [28]. Even though there are solid algorithms for developing material for specialised subjects, combining them without supervision is generally tricky. One way to deal with this is ensuring that the generated material is orthogonal, meaning it has no bearing on the other content. However,

¹<https://www.minecraft.net/en-us/about-minecraft>

²<https://www.mcedit.net/about.html>

³<https://github.com/mcgreentn/GDMC/wiki>

issues remain since the created combination may be improbable, unsuitable, or lacking in variety [30]. A *Minecraft* settlement is an example of such a PCG technique [13].

3. LITERATURE REVIEW

The preceding chapter provided a background discussion of the primary components used in this dissertation. This chapter seeks to provide a more comprehensive review of current state-of-the-art research and applications relevant to the dissertation's goals. It will also further investigate the application of PCG in *Minecraft* settlement generation to determine its effectiveness. Furthermore, we provide a critical analysis of existing methods, where we highlight their strengths and weaknesses.

3.1 City and Road Generation

TownSim [37] is an agent-based city evolution algorithm that gradually creates road networks, inspired by Batty's study [2] on patterns in complicated real-world cities and Emilien et al.'s work [4] on constructing realistic rural-like communities adapted to the steep landscape. Galin, a contributor to [4], has also published a study about a procedural method for generating hierarchical road networks utilising an innovative geometric graph generation algorithm [7]. Furthermore, he was also involved in [19], a study proposing an autonomous approach for creating roadways based on a weighted anisotropic shortest path algorithm. Another system, CityEngine [25], is presented by Parish and Müller, a modified L-system that can simulate an entire city with little data. It develops urban settings from the ground up, based on a hierarchical set of rules that the system may modify based on the demands of the users. Vanegas et al. [43] present an interactive system that includes a geometrical modelling engine consisting of an adaptive road network generator, parcel generator, and building generator. Parcel computation occurs within the blocks established by a road network, whereas building envelopes are constructed procedurally, with their size being determined based on the population and occupations to be housed. Nishida et al. describe a "template-based" interactive application in their study [24] that allows untrained people to construct realistic highways by expanding a geometric graph. There are two sorts of growth procedures used: example-based growth, which utilises patches adopted from the original example to generate routes that maintain structures in the sample road networks, and procedural-based growth, which uses statistical data from the actual instance while adjusting the streets to the underlying terrain and previously generated roads.

3.2 PCG in games

Liapis et al. [17] explore game facet orchestration as the main obstacle for AI-based game generation, defining its dimensions and reviewing research in automated game generation that has aimed to address it. Notably, they distinguish the many creative aspects of games and suggest top-down and bottom-up techniques to orchestration. Values in PCG are explored in [34], where Smith discusses what researchers value in Procedural Generation, mentioning the likes of the *Mario* series and *Zelda*-like games that bring back memories for both players and researchers. He criticises the games we create or utilise as testbeds for research for being constructed around a notion of challenge and difficulty, claiming that these matters pervade the structures of generative sys-

tems.

Yannakaki's work [44] provides a taxonomy of PCG algorithms, which adopt an approach termed *Experience-Driven Procedural Content Generation (EDPCG)*. This technique is broad and adaptable to various HCI sub-areas, with games serving as an example. Smith's taxonomy [33] for PCG deconstructs game and PCG system design characteristics using the MDA framework. This paper discusses numerous reasons for utilising PCG in analogue games, with the mechanics of PCG systems taking centre stage.

The study of "search-based" procedural content generation by Togelius et al. [41] examines the production of game material through genetic algorithms and other optimisation techniques. In [1], they also distinguish between user-produced content and procedural content production, claiming that, despite certain similarities, content created by humans is, ultimately, not procedurally generated. With their focus being on digital games, Hendrikx et al. [12] evaluate the range of procedurally generated content produced and the standard algorithms employed to make it.

Kreminski and Wardrip-Fruin identify two methods to procedural terrain creation and the play dynamics they tend to enable in their work, which they term mining and gardening [16]. As an example, they describe *Minecraft* as a mining game, where the fundamental dynamic of gameplay is extractive, and PCG is employed primarily to generate an infinite amount of content for the player to explore. On the other hand, they characterise *Animal Crossing* as a representation of a gardening game, which encourages the player to engage deeply with a small number of created artefacts and utilise generative methods to gradually change or improve these artefacts over time.

3.3 PCG in Minecraft

Heavily influenced by the constrained rectangular L-growth algorithm discussed by Lopes et al. [18], the building generator discussed in [9] includes a constrained growth method that uses the natural unit provided in *Minecraft*, a single block, to create functionally similar structures with uniquely defined looks. Floor plan generation and exterior wall generation are the two approaches used, with aspects such as room placement, room growth, and door placement all taken into account. Adrian Brightmoore⁴ has showcased his use of PCG in *Minecraft* across a variety of projects, including assisting Blockworks⁵ in the building of the Great Fire of London⁶, recreating a Jawa Sandcrawler, an Armada, and much more.

One may also find existing work regarding the support of procedural generation AI experiments within *Minecraft* on the Project Malmo [15] platform, which establishes the collaborative work done by the ToMCAT project⁷ and the mcg library [27].

⁴<http://www.brightmoore.net/>

⁵<https://www.blockworks.uk/>

⁶<https://www.museumoflondon.org.uk/discover/great-fire-1666>

⁷<https://ml4ai.github.io/tomcat/>

3.4 The GDMC Competition

The GDMC competition⁸, which debuted in 2018, aims to improve generative design [20] and procedural content generation [32] in games, particularly in addressing the hurdles of adaptable and holistic PCG. The main goal for participants consists of creating an AI agent that can compete with humans in the production of practical, visually pleasing, and plausible settlements tailored to a *Minecraft* map. A panel of judges evaluates the submitted solutions based on a large set of criteria to guarantee that assessment is done on all aspects of the problem and challenge participants to create a solution that balances out various objectives [30]. Research is currently being done regarding an aerial cinematographer AI to help with the judging process. The trained AI would film a precise video to mimic the judges' movement and behaviour so that many people can then evaluate the footage appropriately [14]. The Chronicle Challenge [31] is also available, an optional feature for contestants in this competition, where a *Minecraft* book is to be generated with written text concentrating on a narrative based on the history of the constructed settlement.

The first year of this competition [29] included three unseen maps with increasing difficulty alongside four submitted algorithms tested on each map, resulting in 12 total maps containing the generation of settlement structures. The judges recruited included experts with various backgrounds, which observed each map from the players' point of view and then scored each entry based on the adaptability, functionality, evocative narrative, and aesthetics criteria. Results⁹ revealed the strengths and weaknesses of each entry, describing each algorithm below:

3.4.1 Entry 1 - Filip Skwarski - 1st place - 4.38 overall score

Inspired by how humans undergo the construction of settlements, this generator uses a bottom-up approach to populate the map with new structures. Skwarski's algorithm evaluates several buildings to select the best candidate based on the surrounding environment. Upon placement of the structure, the A* pathfinding algorithm [10] adds a new road. It connects it to the closest neighbour, with impassable hill edges that prevent diagonal rising or descending. The result is a collection of residences grouped around fountains and connected by roads, with occasional farms also being present. Although the created structures fit well within the heightmap, they lack interior decoration, diminishing the players' immersion in the game [6].

3.4.2 Entry 2 - Adrian Brightmoore - 3rd place - 2.55 overall score

Mainly operating on a 2d-representation of the map, this submission utilises a two-step approach. Firstly, the heightmap of the provided map is analysed to determine the locations for all of the structures. A comparison is then made between neighbouring structures to determine if there is a difference in height larger than two blocks, which is not jumpable in *Minecraft* and is therefore considered an edge. Finding a region with no 'edges' running across it determines the plot. The generator, originally intended to help build Blockworks'

historic London 'Coding Ye Olde London' project [3], produced the arrangements for this entry. Unlike other entries, structures consisted of interior design, multiple floors, stairs, and windows; yet, the lighting was inadequate. Pathways are not present, but adaptivity was involved by attempting to avoid large cliffs, making the area safer. Overall, the shortage of structures and the absence of infrastructure contributed to a "country living" theme, even though there was no narrative premise to help mould the generator.

3.4.3 Entry 3 - Shaofang Ye and Changxing Cao - 4th place - 2.20 overall score

This submission is motivated by the CGA shape technique [23] and employs a top-down approach. Firstly, the whole area is flattened and divided into plots for the placement of structures, including the use of two procedures: add and cut. The A* algorithm [10] generated a single purple road while ensuring no overlapping with other structures occurred. Despite the lack of a substantial road network [37] due to only one road being present, moving through the generated settlements is simple since the terrain is flat and the buildings are well-positioned relative to one another. As mentioned by the developers, adaptation required further enhancements since the algorithm failed to address the ideal blocks to be used compared to the locally accessible ones. Other improvements are also necessary, as some buildings turned out to be larger or smaller than anticipated, resulting in a less pleasant dwelling. Furthermore, the lack of illumination was also an issue, which is essential for the ease of manoeuvrability and reassurance in the dark [5].

3.4.4 Entry 4 - Rafael Fritsch - 2nd place - 3.08 overall score

In terms of material composition, the main design of the final entry was to adapt to the surroundings. Firstly, the site is analysed to keep a tally of all the occurrences of each block type, and therefore determine the frequencies of various types of wood and stone to aid in selecting the best building materials to be utilised. Afterwards, the algorithm evaluates the area to determine the climatic conditions, with the average temperature being derived using a mix of biome frequencies and information from the *Minecraft* Wiki¹⁰. Furthermore, choosing the appropriate type of crops and vegetation is done by considering the amount of surface water and fertile land available nearby. This technique can also adapt to the ground level, defined by the highest solid block while disregarding plants and other crafted materials. This generator also divides the map into rectangular plots by bisecting the available area repeatedly. Every element is broken into two pieces and then split by a third portion in the middle used for the placement of roads. Following that, it chooses the most appropriate construction based on several 'builders'. A modular method is adopted when constructing the houses. Laying of the foundation is done first, followed by the shell. Finally, doors and windows are positioned appropriately to the room arrangement.

The first year exemplifies a successful competition, with the four contestants sharing their perspectives on how to approach such an issue, and recognising diverse strengths and shortcomings throughout. The competition, also held

⁸<https://gendesignmc.engineering.nyu.edu/>

⁹<https://gendesignmc.engineering.nyu.edu/results>

¹⁰https://minecraft.fandom.com/wiki/Minecraft_Wiki

in 2019 and 2020, saw a rise in participation each year. Additionally, higher overall scores imply a greater understanding of the problem. Table 1 depicts a compilation of the results from both years, summarised from the official results [8].

Entry	Author	Overall Score
2019		
1	Adrian	3.50
2	ArtCodeOutdoors	4.95
3	Berlier	4.70
4	ehauckdo	3.95
5	Julos14	5.39
6	Filip Skwarski	5.50
2020		
1	David Mason	4.42
2	The World Foundry	4.11
3	ICE_JIT	4.38
4	The Charretiers	3.85
5	University of Tsukuba	4.22
6	maelg	3.91
7	Megacruxis	0.00
8	Troy, Ryan, & Trent	4.91
9	Architectural Profiles	2.65
10	GDMC-Minecrafters	0.00
11	Grupo Delu	3.47

Table 1: Table of GDMC results 2019-2020

4. PROPOSED SOLUTION

The proposed solution for this dissertation is a filter written in Python code, executed in a selection box within the MCEdit software. As recommended by the GDMC competition, the artefact will only modify the map within the specified selection box rather than not having any limit. Similar to Berlier’s 2019 GDMC submission¹¹, the intention is to utilise graphs to isolate parts based on the slope of an area. The graph is prepared by first passing the input data through an adaptation system, which determines all the chunks that meet particular requirements and then transform the resulting chunks into a graph. After preparing the graph, graph grammar helps us achieve surface adaptation, where the algorithm modifies the height of the world surface. Subsequently, houses or farmland are the first to be built, depending on how flat the surface is, choosing a particular theme and design depending on the district. Finally, roads are generated using the A* pathfinding algorithm, which draws a route for every edge in the graph while avoiding buildings, water, and steep slopes. This will result in multiple local roads linking to a large global road, completed when all of the dwellings are linked to one another. Furthermore, the algorithm will choose the material of the roads based on the currently designated biome. As a final product, the intended solution would be to have a PCG algorithm capable of generating aesthetic settlements that are both functional and adaptable.

5. EVALUATION PLAN

To assess the data gathered in our project, we propose to incorporate two evaluation procedures: an expressive range

analysis and a user study. An expressive range analysis is a non-user based controlled experiment to assess what the generator can create and recognise how close we came to achieving the envisioned objectives. When deemed adequate, we will subsequently evaluate the developed artefact through a user study designed to simulate the judging done by the individuals in the GDMC competition, where a selection of experienced Minecraft players will serve as a panel of judges and review the system. Judges will begin by being given a variety of maps featuring generated settlements, including our artefact and ones from prior GDMC competitions. The judging process is done while not knowing which map is whose, and the order in which each participant examines maps is randomised. After reviewing a map, they will score between 1 and 10 on each of the following criteria: adaptability, functionality, evocative narrative, and aesthetics. Questions for this user study for each criterion will be adopted from the ones documented in the GDMC paper [30].

6. CONCLUSION

This report introduces Minecraft and PCG, our motivation to work on such a generator, and the objectives we intend to achieve. We provided some background on the technologies to be employed, alongside a comprehensive review of the current state-of-the-art research in our area of interest. Primarily, we discussed City and road generation, PCG in games, and PCG in Minecraft. We also delved deeper into the GDMC competition and showed the different results achieved throughout the years. Finally, we provided an overview of the proposed solution and our evaluation plan, where we intend to conduct a user study to assess our artefact’s adaptability, functionality, evocative narrative, and aesthetics.

7. REFERENCES

- [1] Search-based procedural content generation: A taxonomy and survey. *IEEE Transactions on Computational Intelligence and AI in Games*, 3(3):172–186, Sept. 2011.
- [2] M. Batty. *Cities and Complexity: Understanding Cities with Cellular Automata, Agent-Based Models, and Fractals*. The MIT Press, 2007.
- [3] M. Davies. Coding ye olde london. 2016.
- [4] A. Emilien, A. Bernhardt, A. Peytavie, M.-P. Cani, and E. Galin. Procedural generation of villages on arbitrary terrains. 28(6–8):809–818, June 2012.
- [5] S. Fotios, J. Unwin, and S. Farrall. Road lighting and pedestrian reassurance after dark: A review. *Lighting Research and Technology*, 47, 06 2014.
- [6] J. Freiknecht and W. Effelsberg. Procedural generation of multistory buildings with interior. *IEEE Transactions on Games*, 12(3):323–336, 2020.
- [7] E. Galin, A. Peytavie, E. Guérin, and B. Beneš. Authoring Hierarchical Road Networks. *Computer Graphics Forum*, 30:2021–2030, Sept. 2011.
- [8] M. Green. The GDMC Competition - Results, 2018.
- [9] M. Green, C. Salge, and J. Togelius. Organic building generation in minecraft. pages 1–7, 08 2019.
- [10] P. E. Hart, N. J. Nilsson, and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

¹¹<https://gendesignmc.engineering.nyu.edu/results>

- [11] Hello Games. No man's sky, 2016.
- [12] M. Hendrikx, S. Meijer, J. Velden, and A. Iosup. Procedural content generation for games: A survey. *ACM Transactions on Multimedia Computing, Communications, and Applications (TOMCCAP)*, 9, 02 2013.
- [13] J. Hervé and C. Salge. Comparing PCG metrics with human evaluation in minecraft settlement generation. *CoRR*, abs/2107.02457, 2021.
- [14] H. Jia, R. Thawonmas, and P. Paliyawan. An aerial cinematographer ai for settlements in minecraft—toward their crowd assessment. 2021.
- [15] M. Johnson, K. Hofmann, T. Hutton, and D. Bignell. The malmo platform for artificial intelligence experimentation. In *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence, IJCAI'16*, page 4246–4247. AAAI Press, 2016.
- [16] M. Kreminski and W.-F. Noah. Gardening games : An alternative philosophy of pcg in games. 08 2018.
- [17] A. Liapis, G. Yannakakis, M. Nelson, M. Preuss, and R. Bidarra. Orchestrating game generation. PP:1–1, 09 2018.
- [18] R. Lopes, T. Tutenel, R. Smelik, K. J. de Kraker, and R. Bidarra. A constrained growth method for procedural floor plan generation. 01 2010.
- [19] N. Marechal, E. Galin, A. Peytavié, N. Maréchal, and E. Guérin. Procedural Generation of Roads. *Computer Graphics Forum*, 29:429–438, 2010.
- [20] J. P. McCormack, A. Dorin, and T. C. Innocent. Generative design: A paradigm for design research. *Futureground (Redmond, Durling, de Bono 17November 2004 to 21 November 2004)*, 2(1):0–0, 2005.
- [21] Mojang. Minecraft classic, 2009.
- [22] Mojang. Minecraft, 2011.
- [23] P. Müller, P. Wonka, S. Haegler, A. Ulmer, and L. Van Gool. Procedural modeling of buildings. 25(3):614–623, 07 2006.
- [24] G. Nishida, I. Garcia-Dorado, and D. Aliaga. Example-driven procedural urban roads. *Computer Graphics Forum*, 35, 09 2015.
- [25] Y. I. H. Parish and P. Müller. Procedural modeling of cities. In *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques, SIGGRAPH '01*, page 301–308, New York, NY, USA, 2001. Association for Computing Machinery.
- [26] A. Pyarelal, A. Banerjee, and K. Barnard. Modular procedural generation for voxel maps. *CoRR*, abs/2104.08890, 2021.
- [27] A. Pyarelal, A. Banerjee, and K. Barnard. Modular procedural generation for voxel maps. *CoRR*, abs/2104.08890, 2021.
- [28] C. Salge, C. Aranha, A. Brightmoore, S. Butler, R. Canaan, M. Cook, M. C. Green, H. Fischer, C. Guckelsberger, J. Hadley, J. Hervé, M. R. Johnson, Q. Kybartas, D. Mason, M. Preuss, T. Smith, R. Thawonmas, and J. Togelius. Impressions of the GDMC AI settlement generation challenge in minecraft. *CoRR*, abs/2108.02955, 2021.
- [29] C. Salge, M. Green, R. Canaan, F. Skwarski, R. Fritsch, A. Brightmoore, S. Ye, C. Cao, and J. Togelius. The ai settlement generation challenge in minecraft: First year report. *KI - Künstliche Intelligenz*, 34, 01 2020.
- [30] C. Salge, M. Green, R. Canaan, and J. Togelius. Generative design in minecraft (gdmc): settlement generation competition. In S. Deterding, M. Khandaker, S. Risi, J. Font, S. Dahlskog, C. Salge, and C. Olsson, editors, *Proceedings of the 13th International Conference on the Foundations of Digital Games, FDG 2018*, ACM International Conference Proceeding Series. Association for Computing Machinery, 08 2018.
- [31] C. Salge, C. Guckelsberger, M. Green, R. Canaan, and J. Togelius. Generative design in minecraft: Chronicle challenge. 05 2019.
- [32] N. Shaker, J. Togelius, and M. J. Nelson. *Procedural Content Generation in Games*. Springer Publishing Company, Incorporated, 1st edition, 2016.
- [33] G. Smith. Understanding procedural content generation: A design-centric analysis of the role of pcg in games. *Conference on Human Factors in Computing Systems - Proceedings*, 04 2014.
- [34] G. Smith. What do we value in procedural content generation? In *Proceedings of the 12th International Conference on the Foundations of Digital Games, FDG '17*, New York, NY, USA, 2017. Association for Computing Machinery.
- [35] W. Software. Factorio, 2016.
- [36] S. E. Softworks. Astroneer, 2016.
- [37] A. Song and J. Whitehead. Townsim: agent-based city evolution for naturalistic road network generation. pages 1–9, 08 2019.
- [38] I. Studios. Valheim, 2021.
- [39] A. Szlam, J. Gray, K. Srinet, Y. Jernite, A. Joulin, G. Synnaeve, D. Kiela, H. Yu, Z. Chen, S. Goyal, D. Guo, D. Rothermel, C. L. Zitnick, and J. Weston. Why build an assistant in minecraft?, 2019.
- [40] J. Togelius, A. Champandard, P. L. Lanzi, M. Mateas, A. Paiva, M. Preuss, and K. Stanley. Procedural content generation: goals, challenges and actionable steps. *Dagstuhl Follow-Ups*, 6:61–75, 01 2013.
- [41] J. Togelius, E. Kastbjerg, D. Schedl, and G. Yannakakis. What is procedural content generation? mario on the borderline. In *2nd International Workshop on Procedural Content Generation in Games, PCGames 2011 - Co-located with the 6th International Conference on the Foundations of Digital Games*, ACM International Conference Proceeding Series, 01 2011.
- [42] M. Toy, G. Wichman, K. Arnold, and J. Lane. Rogue, 1980.
- [43] C. Vanegas, D. Aliaga, B. Benes, and P. Waddell. Interactive design of urban spaces using geometrical and behavioral modeling. *ACM Transactions on Graphics*, 28:10, 12 2009.
- [44] G. Yannakakis and J. Togelius. Experience-driven procedural content generation. *Affective Computing, IEEE Transactions on*, 2:147–161, 07 2011.